

Literature Review and Evaluation of an AIoT-Based Predictive Maintenance Project Using a Classification Model

Date: 19 April 2026

Abstract

This paper presents a research-paper-style literature review for an **AIoT-based Predictive Maintenance** project in which the core machine learning approach is a **classification model**. The review focuses on how Industrial Internet of Things infrastructures, real-time sensing, data preprocessing, and classification-oriented machine learning models can be combined to identify equipment states, fault categories, or warning conditions before critical breakdowns occur. The paper synthesizes evidence from peer-reviewed studies and complements it with selected GitHub implementations to bridge theory and practice. In addition to the literature review, the paper includes a lecturer-oriented evaluation section, a set of suggested improvements tailored to a classification-based system, and a proposed grading rubric covering documentation, implementation, testing, and presentation. The review concludes that classification is a strong and practical starting point for student predictive maintenance projects because it supports interpretable operational decisions such as **normal**, **degraded**, and **fault** state recognition, while remaining extensible to future tasks such as anomaly detection and Remaining Useful Life estimation.[1](#) [3](#)

1. Introduction

Predictive maintenance has emerged as one of the most important applications of artificial intelligence in smart manufacturing because it shifts maintenance from a reactive or schedule-based activity to a data-driven decision process. In AIoT environments, machines are equipped with connected sensors that continuously

monitor physical variables such as temperature, vibration, pressure, and current. These measurements are then transmitted through communication and storage layers to analytics pipelines that infer equipment health and trigger maintenance actions before a severe failure happens.[1](#)

For a student project, a **classification model** is often more suitable than a regression-oriented Remaining Useful Life model because classification directly supports operational maintenance questions such as whether a machine is in a healthy condition, entering a warning zone, or already exhibiting a recognizable fault type. This formulation is also easier to evaluate through confusion matrices, precision, recall, and F1-score, which makes the system easier to explain during academic assessment and presentation.[3](#) [6](#)

“Predictive maintenance is a crucial component of smart manufacturing in Industry 4.0, utilizing data from IoT sensor networks and machine learning algorithms to predict equipment failures before they happen.” — Rakholia et al., 2025.[1](#)

2. Conceptual Background: AIoT and Classification-Based Predictive Maintenance

An AIoT predictive maintenance system combines three layers of intelligence. The first layer is the **physical sensing layer**, where industrial assets produce real-time operational signals through IoT sensors. The second layer is the **data and communication layer**, where streams are collected, cleaned, synchronized, and integrated with contextual data such as maintenance logs or ERP production records. The third layer is the **AI layer**, where machine learning models transform signals into decisions, such as classifying whether the machine is operating normally or which failure mode is emerging.[1](#) [5](#)

In the classification setting, the target variable is categorical rather than continuous. Typical labels may include **normal**, **fault**, **warning**, or more specific categories such as motor overheating, feeder stop, bearing anomaly, or vibration threshold exceedance. This framing is especially appropriate when the maintenance team needs immediate actionable categories rather than a precise remaining lifetime estimate. It also fits industrial datasets where fault annotations exist as discrete events or alarm classes.[3](#)

Dimension	Classification-Based PdM	Regression/RUL-Based PdM
Main output	Discrete class label	Continuous time or health estimate
Typical question	What is the machine state or fault type?	How long until failure?
Common metrics	Accuracy, Precision, Recall, F1, ROC-AUC	RMSE, MAE, Score functions
Project suitability for students	High, due to clearer interpretation	Moderate to advanced
Operational action	Alarming and fault categorization	Planning long-horizon maintenance

3. Literature Review

3.1 System-Level Role of AIoT in Predictive Maintenance

Recent literature consistently shows that predictive maintenance effectiveness depends not only on the machine learning algorithm but also on the integration of sensing, communication, preprocessing, and decision support. Rakholia et al. describe predictive maintenance in Industry 4.0 as an integrated process where IoT sensor data are combined with production and ERP information to improve operational context and short-horizon prediction quality.[1](#) Their study is particularly relevant to a student classification project because it demonstrates that maintenance intelligence improves when raw sensor signals are enriched with production-side information instead of being treated as isolated measurements.

Ucar et al. further clarify that AI-based predictive maintenance systems are built from interdependent components, including sensors, preprocessing, AI algorithms, decision modules, communication and integration mechanisms, and reporting dashboards.[2](#) This system-level view is highly valuable for an AIoT project because it shows that a successful model is not merely an isolated classifier. Rather, it is part of a broader operational loop that begins with reliable sensing and ends with maintainable, interpretable, and actionable maintenance decisions.

3.2 Evidence from Systematic Reviews

A major systematic review by Haque et al. synthesizes 78 peer-reviewed studies and shows that AI-driven predictive maintenance has matured into a central strategy in industrial automation.³ The review reports that IoT-enabled condition monitoring, edge-cloud integration, and AI-based models collectively improve prediction quality, reduce maintenance cost, and increase equipment uptime. For the present project, the most important implication is that the value of classification models is not limited to raw prediction accuracy; it lies in their role as decision tools embedded in a wider sensing and maintenance ecosystem.

The same review also identifies three persistent barriers: **data quality issues**, **cybersecurity concerns**, and **limited interpretability of advanced models**.³ These challenges are directly relevant to a student project. In academic settings, datasets are often simulated, partially labeled, or imbalanced; therefore, choosing a classification formulation can help stabilize the project by making the output space clearer and the evaluation more transparent. At the same time, the project should explicitly discuss trustworthiness, explainability, and data governance if it is to be judged as a serious AIoT engineering system rather than a standalone coding exercise.²

3.3 Classification Models in the Literature

Among the most relevant studies for this project is the work of Elkateb et al., who proposed a machine learning and IoT-based predictive maintenance approach for industrial applications and used **AdaBoost** to classify six types of machine stops in knitting machines.⁴ The system relied on machine speed and step data acquired from connected equipment and achieved approximately **92% test accuracy**, showing that classification is not only academically valid but also practically effective for industrial stop-category identification.⁴ This study strongly supports the student's choice of a classification model because it demonstrates a complete pathway from sensed data to actionable maintenance categories.

Beyond AdaBoost, the broader literature identifies **Logistic Regression**, **XGBoost**, **Random Forest**, **CNN**, and **LSTM** as important candidates depending on the structure of the dataset and the temporal nature of the signals.^{1 3} In a student project, the selection should be driven by the availability of labeled classes, class balance, explainability requirements, and whether the data are treated as aggregated tabular features or sequential time series windows. A baseline linear classifier such as Logistic

Regression is valuable for interpretability, while boosted trees and deep learning methods may improve predictive power when the dataset is sufficiently rich.[2](#) [6](#)

Study / Source	IIoT Context	Model Type	Task Type	Key Contribution
Rakholia et al. (2025) 1	IIoT + ERP integration in Industry 4.0	XGBoost	Short-horizon fault-related prediction	Demonstrates integration of sensor and enterprise data
Haque et al. (2024) 3	Systematic review of industrial automation	CNN, LSTM and other AI models	Review	Summarizes trends, benefits, and research gaps
Elkateb et al. (2024) 4	IIoT-enabled industrial monitoring	AdaBoost	Classification	Classifies six machine stop types with strong accuracy
Ucar et al. (2024) 2	AI-based PdM architecture	Multiple AI methods	Review	Defines system components and trustworthiness issues
IBM GitHub project 6	Time-stamped IIoT sensor analytics	Logistic Regression	Classification	End-to-end practical workflow with confusion matrix

3.4 Practical Insights from GitHub Projects

GitHub repositories are not substitutes for peer-reviewed evidence, but they are valuable as implementation references because they reveal how IIoT concepts are operationalized in software pipelines. The repository **IBM/iiot-predictive-analytics** provides an end-to-end pattern for predicting equipment failures using time-stamped IIoT sensor data, configurable training pipelines, and confusion-matrix-based evaluation.[6](#) For a student project using classification, this repository is especially useful because it demonstrates how to structure the workflow from data access to model scoring and evaluation.

The repository **karimosman89/iiot-predictive-maintenance** offers a broader streaming-oriented IIoT architecture that includes Kafka for real-time transport, Spark for processing, AWS IIoT for cloud connectivity, and MLflow for experiment tracking.[5](#)

While this repository is more infrastructure-heavy than a typical graduation assignment, it is a strong practical reference for discussing scalability, deployment readiness, and data engineering maturity.

A third repository from **AWS Labs** focuses primarily on Remaining Useful Life prediction using a Bidirectional LSTM model, which is technically closer to regression than classification.[7](#) Nevertheless, it remains relevant as a future extension pathway because it shows how a student classification project could later evolve into a more advanced prognostics platform integrating scheduled inference, cloud storage, and dashboard-ready outputs.[7](#)

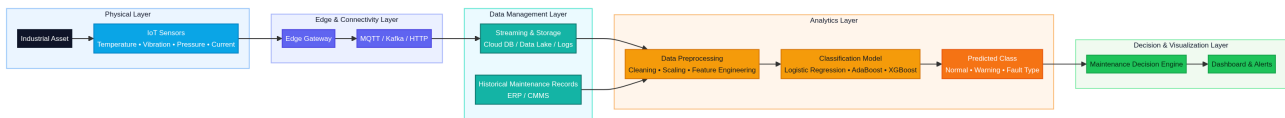
3.5 Research Gaps Relevant to the Student Project

Although the literature demonstrates clear progress, several gaps remain that are directly relevant to this project. First, many studies emphasize algorithmic performance while underreporting real deployment constraints such as missing data, latency, concept drift, and cybersecurity.[2](#) Second, there is a recurring disconnect between highly accurate research models and maintainable industrial systems. A project that explicitly documents its sensing assumptions, labeling strategy, and integration design will therefore be stronger academically.

Third, classification-based predictive maintenance projects often stop at overall accuracy, even though maintenance decisions are usually more sensitive to **false negatives** than false positives. A machine incorrectly labeled as healthy may lead to costly downtime, whereas an unnecessary warning may only lead to extra inspection. Therefore, the literature suggests that class-wise recall, confusion matrix interpretation, and threshold tuning should be central to evaluation rather than optional add-ons.[2](#) [6](#)

4. Proposed Architecture for the Student Project

The most appropriate architecture for the student project is a layered AIoT pipeline in which sensor signals are collected from industrial equipment, transferred through an edge or gateway node, stored or streamed into a preprocessing module, and then fed into a classification model that predicts the equipment state or fault category. The model output should be connected to a dashboard and alerting mechanism so that the project clearly demonstrates operational value rather than only offline analytics.[1](#)



A practical student implementation can begin with a tabular feature approach. In this design, time-window statistics such as mean vibration, standard deviation, maximum temperature, pressure trend, or current imbalance are computed and passed to classifiers such as Logistic Regression, Random Forest, AdaBoost, or XGBoost. If the dataset is richer and time-sequential, the system can later be extended to one-dimensional CNN or LSTM architectures. The essential point is that the first version should remain interpretable, testable, and aligned with the available labels.[2](#) [4](#)

Architectural Layer	Recommended Role in the Project	Suggested Student-Level Choice
Sensor Layer	Capture machine condition signals	Temperature, vibration, current, pressure
Communication Layer	Move data to processing unit	MQTT, HTTP, or simulated streaming
Storage Layer	Persist raw and processed data	CSV, SQLite, Firebase, or cloud bucket
Preprocessing Layer	Cleaning, scaling, feature extraction	Python pipeline with pandas/sklearn
Modeling Layer	Predict machine state	Logistic Regression, AdaBoost, XGBoost
Decision Layer	Convert prediction to action	Warning thresholds and maintenance alerts
Interface Layer	Present results clearly	Dashboard or report with charts

5. Lecturer Feedback and Evaluation

From an academic perspective, this project idea is strong because it combines **machine learning**, **IoT sensing**, and **industrial relevance** in a single coherent application. The choice of predictive maintenance is appropriate for an AIoT course or graduation project because it demonstrates interdisciplinary understanding across

embedded sensing, data engineering, machine learning, and systems integration. Moreover, choosing a **classification model** is pedagogically sound because it allows the student to present a problem that is easy to interpret and assess, especially when the system predicts discrete machine conditions or fault types.[3](#) [6](#)

The project would likely be evaluated positively if it clearly shows the full pipeline from data acquisition to maintenance decision. A lecturer would particularly appreciate evidence that the student understands why features were chosen, why a classification formulation was adopted, how class labels were defined, and how the evaluation metrics relate to real maintenance risk. If the project includes a dashboard or clear monitoring output, it will further strengthen the impression that the work goes beyond theory into deployable AIoT thinking.[1](#) [5](#)

However, a lecturer may also raise critical questions. These may include whether the data are real or simulated, whether the labeling process is reliable, whether class imbalance was handled correctly, and whether the chosen model is interpretable enough for engineering use. If these issues are ignored, the project may appear technically attractive but scientifically incomplete. Therefore, documentation quality and critical discussion are as important as raw model accuracy.[2](#)

6. Suggested Improvements

The first recommended improvement is to establish a clearer **labeling strategy**. Since the project uses classification, the classes should be explicitly justified. For example, the student may define three classes such as **healthy**, **warning**, and **fault**, or may define several fault categories according to the mechanical context. The labels must be derived from operational thresholds, maintenance logs, or domain-inspired simulated scenarios rather than chosen arbitrarily.

The second improvement is to strengthen **evaluation rigor**. Instead of relying only on accuracy, the project should report precision, recall, F1-score, and a confusion matrix. If the classes are imbalanced, macro-averaged and weighted scores should also be included. This is especially important in predictive maintenance because the cost of missing a real fault is often larger than the cost of issuing a false alarm.[2](#) [6](#)

The third improvement is to enhance **system realism** through AIoT integration. Even if the sensing is simulated, the project should show a realistic pipeline involving sensor generation, streaming or periodic ingestion, storage, preprocessing, model inference,

and alert visualization. References from GitHub indicate that even a student-scale project becomes much stronger when the modeling stage is embedded within a data pipeline rather than presented as a single notebook.[5](#)

The fourth improvement is to address **trustworthiness and explainability**. Feature importance analysis, SHAP-style explanation, or at least coefficient interpretation in a baseline model would help explain why the system predicts a given class. This aligns with current research trends emphasizing trustworthy AI in predictive maintenance and will make the project more convincing during oral defense.[2](#)

The fifth improvement is to state a **future roadmap**. After presenting a successful classification system, the student can explain how the project could later extend to anomaly detection, fault severity estimation, or Remaining Useful Life prediction. This shows technical maturity and awareness of how classification fits within the wider predictive maintenance research landscape.[2](#)

7. Final Grading Criteria

The following grading rubric is designed to align with the expected strengths of a classification-based AIoT predictive maintenance project. It balances scientific writing, engineering implementation, empirical testing, and communication quality.

Criterion	Weight	What the Lecturer Should Look For
Documentation and Literature Review	25%	Clear problem statement, academic structure, use of recent papers, proper citations, and justified model choice
System Design and Implementation	30%	End-to-end AIoT pipeline, data flow clarity, preprocessing quality, correct use of classification algorithms, and code organization
Testing and Evaluation	25%	Appropriate train/test strategy, confusion matrix, precision, recall, F1-score, handling of imbalance, and discussion of limitations
Presentation and Demonstration	20%	Clear explanation, professional visuals, strong defense of decisions, and convincing live demo or results walkthrough

A more detailed mark interpretation can also be used to support transparent assessment.

Performance Band	Indicative Mark	Interpretation
Excellent	90–100	Complete AIoT pipeline, strong literature basis, robust classification results, critical discussion, and excellent presentation
Very Good	80–89	Well-implemented system with good evaluation and documentation, but minor limitations in depth or deployment realism
Good	70–79	Functional project with acceptable classification model and moderate documentation, though analysis remains somewhat limited
Satisfactory	60–69	Basic model and simple implementation, but limited AIoT integration, weak evaluation, or insufficient discussion
Weak	Below 60	Incomplete pipeline, poor documentation, inadequate testing, or unclear understanding of predictive maintenance concepts

8. Conclusion

This literature review shows that an AIoT-based predictive maintenance project built around a **classification model** is academically justified and practically meaningful. The literature demonstrates that predictive maintenance depends on the integration of IoT sensing, preprocessing quality, machine learning, and decision support rather than on the classifier alone.[1](#) [3](#) Studies such as Elkateb et al. confirm that classification can effectively identify operational stop categories in industrial settings, while systematic reviews and architectural surveys highlight the importance of trustworthiness, data quality, and deployment awareness.[2](#) [4](#)

For the present project, the strongest strategy is to build a clear and well-documented system that classifies machine state or fault category using interpretable inputs and rigorous evaluation. If executed carefully, such a project can earn strong academic feedback because it demonstrates not only a working model, but also engineering

judgment, research awareness, and a credible path toward more advanced predictive maintenance capabilities.[5](#) [7](#)

References

1. Rakholia, R., Suárez-Cetrulo, A. L., Singh, M., and Darb, R. S. C. (2025). *Integrating AI and IoT for Predictive Maintenance in Industry 4.0 Manufacturing Environments: A Practical Approach*. Information, 16(9), 737. DOI: [10.3390/info16090737](https://doi.org/10.3390/info16090737).
2. Ucar, A., Karakose, M., and Kırımça, N. (2024). *Artificial Intelligence for Predictive Maintenance Applications: Key Components, Trustworthiness, and Future Trends*. Applied Sciences, 14(2), 898. DOI: [10.3390/app14020898](https://doi.org/10.3390/app14020898).
3. Haque, R., Bajwa, A., Siddiqui, N. A., and Ahmed, I. (2024). *Predictive Maintenance in Industrial Automation: A Systematic Review of IoT Sensor Technologies and AI Algorithms*. American Journal of Interdisciplinary Studies, 5(01), 1–30. DOI: [10.63125/hd2ac988](https://doi.org/10.63125/hd2ac988).
4. Elkateb, S., Métwalli, A., Shendy, A., and Abu-Elanien, A. E. B. (2024). *Machine Learning and IoT-Based Predictive Maintenance Approach for Industrial Applications*. Alexandria Engineering Journal, 88, 298–309. Available at: [ScienceDirect](https://www.sciencedirect.com/science/article/pii/S1110019224000000).
5. karimosman89. (n.d.). *iot-predictive-maintenance*. GitHub repository. Available at: <https://github.com/karimosman89/iot-predictive-maintenance>.
6. IBM. (n.d.). *iot-predictive-analytics*. GitHub repository. Available at: <https://github.com/IBM/iot-predictive-analytics>.
7. AWS Labs. (n.d.). *predictive-maintenance-using-machine-learning*. GitHub repository. Available at: <https://github.com/aws-labs/predictive-maintenance-using-machine-learning>.